



Fundamentos de Bases de Datos y Consultas Básicas

Clase 3 - Modelo de datos y estructura



¿Qué aprenderás en esta sesión?

Aprenderemos a identificar y describir los elementos esenciales de un modelo de datos, incluyendo entidades, atributos e identificadores.

También aprenderemos a representar relaciones entre entidades utilizando diagramas entidad-relación, con el fin de estructurar de forma lógica y coherente una base de datos relacional desde su diseño inicial.

Desafío inicial

Una empresa de educación técnica quiere implementar un sistema de gestión académica. El equipo necesita diseñar la estructura de la base de datos desde cero, pero no tienen claridad sobre cómo organizar la información: ¿qué elementos deben considerarse?, ¿cómo identificar las entidades clave?, ¿cómo se relacionan entre sí?, ¿cómo se representan gráficamente para entender y validar el diseño?

¿Cómo se construye un modelo de datos relacional que represente correctamente la estructura de información de un sistema académico?



Para resolverlo deberás:

1

Reconocer los elementos que componen un modelo de datos.

2

Identificar entidades, atributos e identificadores en base a requerimientos reales.

3

Representar relaciones mediante un diagrama entidad-relación (DER).

Al finalizar esta sesión deberás estar en condiciones de analizar un problema del mundo real, abstraer sus componentes clave, y proponer una estructura de base de datos inicial que sea clara, coherente y funcional, lista para ser validada con el equipo de desarrollo.

Elementos de un modelo de datos

¿Qué es un modelo de datos?

Un modelo de datos es una representación lógica y estructurada de la información relevante para un sistema o proceso organizacional. Su propósito es organizar y documentar cómo se almacenan, agrupan y relacionan los datos, facilitando la comprensión, implementación y mantenimiento de una base de datos.

Los modelos de datos son fundamentales en entornos reales donde la calidad y consistencia de la información impactan directamente la operación del negocio. Por ejemplo, en un sistema de salud, una mala estructuración del modelo puede dificultar el seguimiento de tratamientos, historial clínico o facturación de pacientes.



Existen distintos tipos de modelos de datos:

Modelo conceptual

Esboza una visión general de la estructura de los datos, sin detalles técnicos, ideal para discutir con personas no técnicas.

Modelo lógico

Define entidades, atributos y relaciones de manera detallada, usando conceptos normalizados.

Modelo físico

Detalla cómo se implementa el modelo lógico en un SGBD específico, con tipos de datos concretos, índices, restricciones y configuraciones.



Contexto productivo

Un modelo de datos bien diseñado permite detectar duplicidades, asegurar integridad referencial, optimizar procesos de consulta y respaldar decisiones estratégicas. Además, se convierte en un insumo clave para la automatización de reportes y validación de reglas de negocio.

Componentes esenciales

Los principales elementos que componen un modelo de datos relacional son:

Elemento	Descripción
Entidad	Representa un objeto real o concepto relevante (por ejemplo, Estudiante).
Atributo	Propiedad que describe a una entidad (ej. nombre, fecha_nacimiento).
Identificador	Atributo que distingue de forma única a cada instancia de una entidad.
Relación	Vínculo entre dos o más entidades (ej. un estudiante se matricula en un curso).

Tabla 1. Elementos esenciales de un modelo de datos.

Fuente: Desafío Latam

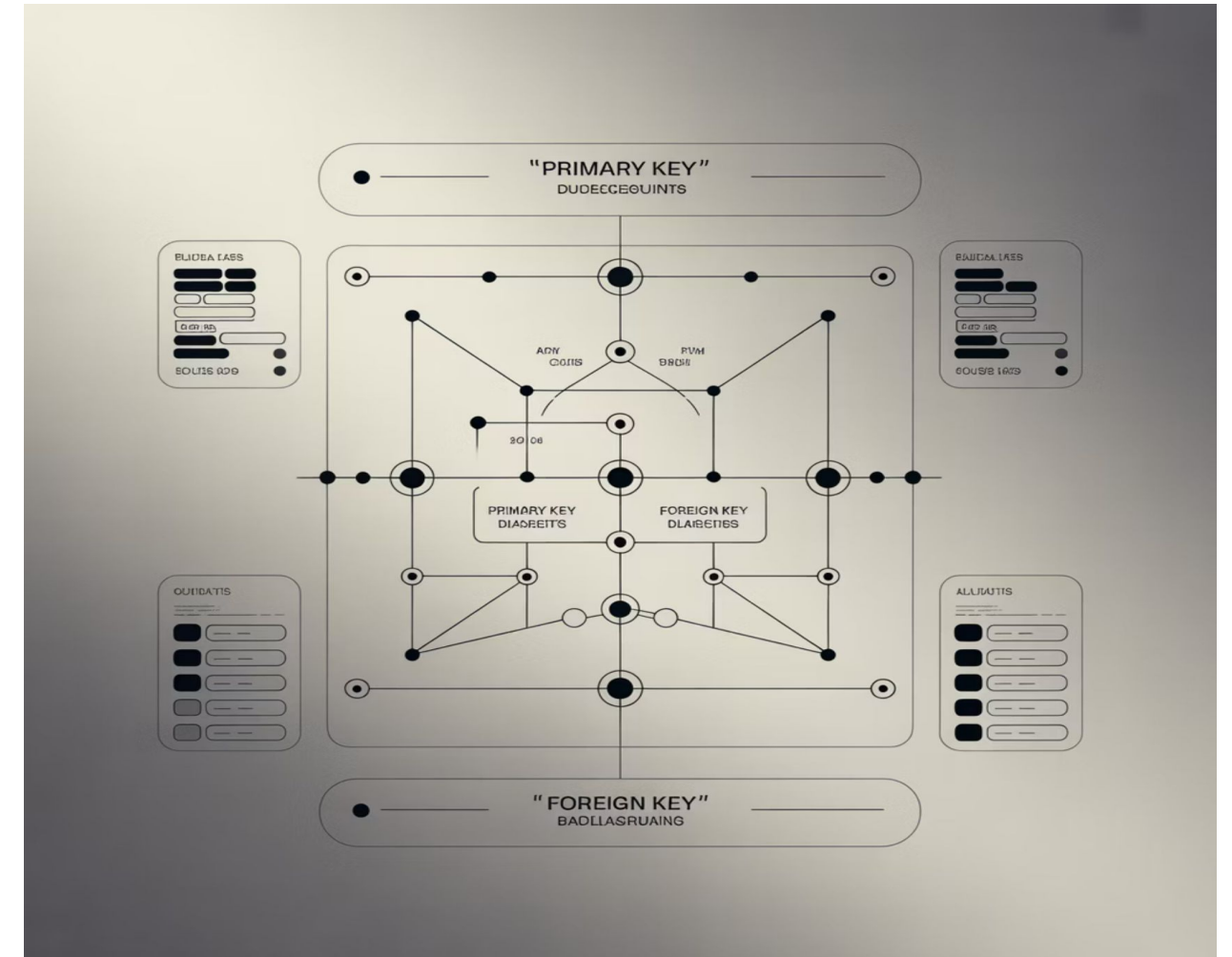
Además, podemos complementar estos elementos con:

Cardinalidad

Determina cuántas veces una entidad puede estar asociada a otra (uno a uno, uno a muchos, muchos a muchos).

Restricciones

Reglas de negocio que condicionan el comportamiento de los datos (por ejemplo, una matrícula no puede existir si no hay un estudiante registrado).



Buenas prácticas aplicadas



Validar los requerimientos funcionales con las personas usuarias antes de definir entidades.



Evitar sobrecargar entidades con atributos innecesarios o ambiguos.



Establecer convenciones de nombres desde el diseño inicial para facilitar la legibilidad y mantenimiento.

Errores comunes en el diseño inicial



Asumir relaciones sin confirmación del equipo de negocio.



Crear entidades demasiado genéricas que luego dificultan la trazabilidad de los datos.



Usar atributos redundantes que pueden generar inconsistencias.

Ejemplo aplicado

En el contexto de una base de datos académica, podríamos tener el siguiente desglose:

Entidad: Estudiante

- Atributos: id_estudiante (identificador), nombre, correo, fecha_nacimiento

Entidad: Curso

- Atributos: id_curso (identificador), nombre, horas, docente_responsable

Entidad: Matrícula (relación entre estudiante y curso)

- Atributos: id_matricula, id_estudiante, id_curso, fecha_inicio



Representación visual simplificada

Estudiante (id_estudiante, nombre, correo)

Curso (id_curso, nombre, horas)

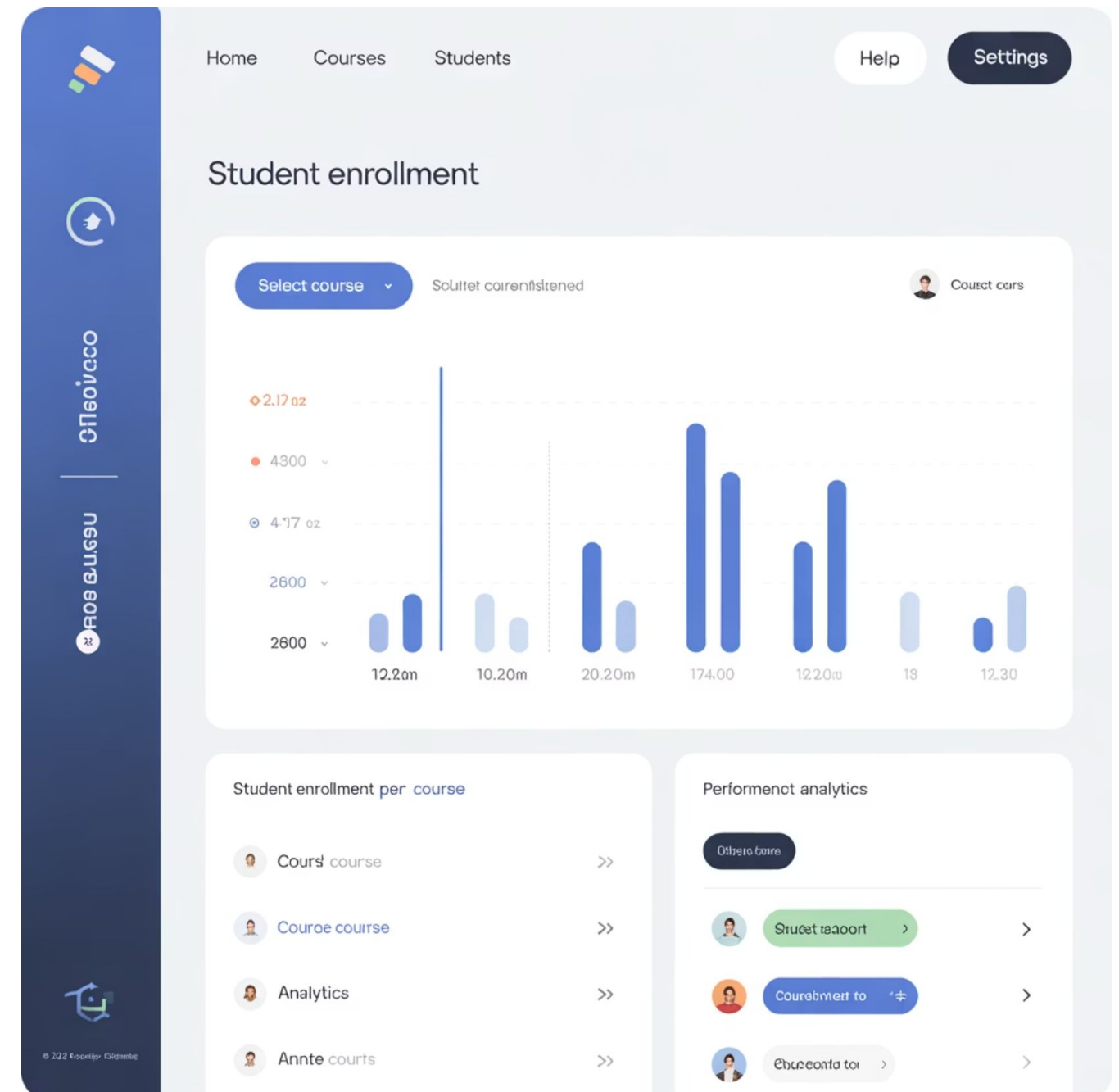
Matrícula (id_matricula, id_estudiante, id_curso, fecha_inicio)

Esta estructura permite representar qué estudiante se ha matriculado en qué curso, en qué fecha y bajo qué condiciones. Además, puede expandirse fácilmente con nuevas entidades como "Docente", "Asignatura" o "Evaluaciones".

Aplicación en entorno real

Este tipo de estructura se utiliza en plataformas educativas (como Moodle, Blackboard o sistemas internos) donde se requiere tener trazabilidad sobre la actividad académica, inscripciones y generación de reportes.

Al comprender los elementos esenciales de un modelo de datos y cómo estos se estructuran en función del contexto, estamos preparados para avanzar hacia el diseño detallado de las entidades, donde definiremos sus atributos, identificadores y su rol dentro de la base de datos relacional.





Entidades, atributos e identificadores

¿Cómo puedes identificar y diferenciar las entidades, atributos e identificadores en los requerimientos de una organización?

¿Qué es una entidad?

Una entidad es una representación de un objeto real o abstracto del que se requiere almacenar información en un sistema. En un modelo de datos, las entidades corresponden a las tablas principales de la base de datos.

Ejemplos comunes de entidades: Persona, Producto, Curso, Cliente, Vehículo, Proveedor, Factura.



Criterios para identificar una entidad



Tiene existencia propia en el dominio del sistema.



Es relevante para las funciones del negocio.



Debe conservar información persistente (no transitoria).



Participa activamente en procesos de negocio (ventas, matrículas, órdenes, etc.).



Puede relacionarse con otras entidades.

Contexto productivo

En un sistema de recursos humanos, una entidad típica es "Empleado". Esta entidad permite registrar información como contrato, jornada laboral, asignaciones, licencias y vínculos con otras áreas como finanzas o capacitación. Un mal diseño de esta entidad puede causar errores en nómina, informes duplicados o problemas de cumplimiento legal.

Buenas prácticas al definir entidades

Reunir a las personas usuarias clave

Para validar las entidades propuestas.

Evitar múltiples conceptos

Una entidad no debe representar múltiples conceptos.

Nombrar en singular

Nombrar las entidades en singular (ej. Cliente, no Clientes).

Errores comunes

- Crear una entidad para cada formulario, sin analizar si los datos ya pertenecen a otra entidad.
- Modelar datos que no son persistentes (ej. "búsqueda actual" como entidad).

Atributos: características de una entidad

Los atributos describen las propiedades de una entidad. Cada atributo se convierte en una columna dentro de una tabla en la base de datos.

Ejemplo: Para la entidad Estudiante, podríamos tener los siguientes atributos:

- nombre_completo
- fecha_nacimiento
- correo_electronico
- carrera
- nivel_educacional
- telefono_contacto
- activo (booleano que indica si el estudiante está vigente)

Tipos de atributos



Simple

No se pueden descomponer (ej. edad).



Compuestos

Pueden dividirse en subcomponentes (ej. nombre completo).



Derivados

Se obtienen a partir de otros datos (ej. edad derivada de fecha de nacimiento).



Multivaluados

Pueden tener más de un valor (por ejemplo, varios correos), pero deben evitarse en modelos relacionales puros.

Contexto productivo

Una definición adecuada de atributos permite implementar validaciones automáticas, facilitar la generación de reportes y habilitar búsquedas eficientes. Por ejemplo, definir un atributo "estado_civil" como tipo de dato restringido puede ayudar a estandarizar encuestas y registros.

Buenas prácticas



Incluir sólo atributos necesarios y relevantes para los procesos del negocio.



Definir correctamente los tipos de datos (texto, fecha, booleano, entero, etc.).



Validar con los equipos de TI y usuarios si se requieren campos obligatorios o valores por defecto.

Errores comunes



Incluir atributos redundantes (como edad y fecha de nacimiento simultáneamente sin justificación).



Usar tipos de datos incorrectos (ej. almacenar fechas como texto).



No considerar atributos clave para futuras integraciones con otros sistemas.

Identificadores: clave primaria y unicidad

El identificador (o clave primaria) es un atributo —o conjunto de atributos— que permite diferenciar de forma única a cada instancia de una entidad. Es esencial para garantizar la integridad de los datos.

Características clave

- Debe ser único.
- No debe cambiar con el tiempo.
- No debe ser nulo.



Tipos de identificadores

Naturales

Derivan de información existente (ej. RUT, número de serie).

Artificiales

Generados por el sistema (ej. id numérico autoincremental).

Ejemplo aplicado: Para la entidad Estudiante:

Atributo identificador: id_estudiante (valor numérico generado por el sistema).

Representación visual

Entidad: ESTUDIANTE

| id_estudiante (PK) |

| nombre_completo |

| fecha_nacimiento |

| correo_electronico |

| carrera |

Contexto productivo

El uso de identificadores artificiales facilita la migración de datos, escalabilidad y permite cambios en los atributos naturales sin afectar la lógica del sistema. Por ejemplo, si un cliente cambia su RUT, su id_cliente permanece intacto.

Buenas prácticas



Elegir identificadores artificiales cuando la lógica de negocio puede cambiar con el tiempo.



Garantizar que los identificadores se indexen para mejorar el rendimiento de consultas.



Evitar claves compuestas en tablas que crecerán rápidamente o se consultarán con frecuencia.

Errores comunes



Usar campos como nombres o correos como claves primarias.



No validar unicidad en sistemas distribuidos.

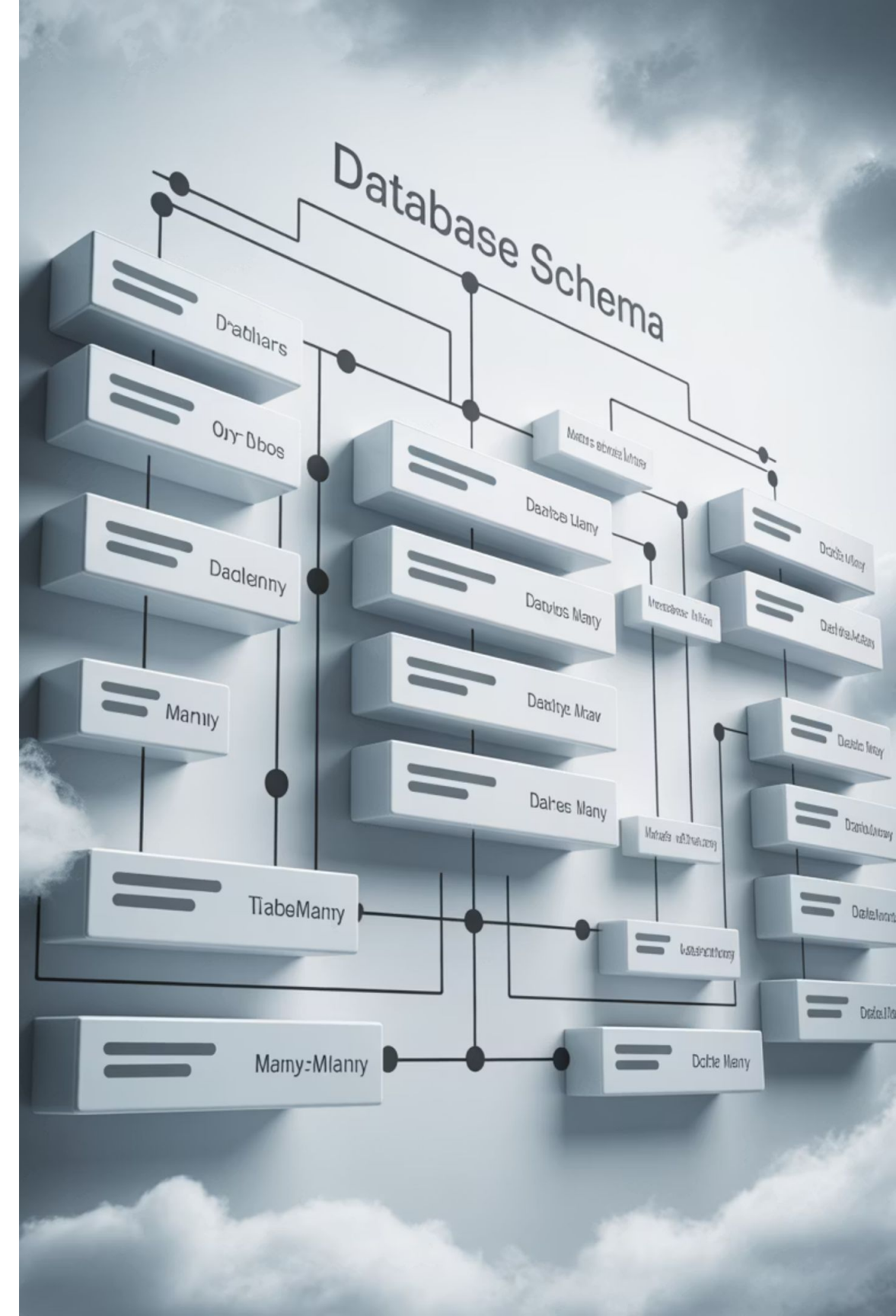


Reutilizar identificadores en sistemas donde los registros eliminados deben conservarse.

Hemos profundizado en la identificación de entidades, atributos e identificadores, elementos fundamentales para diseñar estructuras robustas. El siguiente paso es comprender cómo estas entidades se relacionan entre sí mediante distintos tipos de relaciones, lo que nos permitirá construir una representación gráfica completa con el diagrama entidad-relación.

¿Cómo puedes representar gráficamente las interacciones entre distintas entidades de una base de datos relacional?

Las relaciones permiten establecer cómo se vinculan dos o más entidades dentro del modelo de datos. Estas relaciones representan hechos del negocio, como por ejemplo que un estudiante se matricula en un curso o que un cliente realiza una compra.



Tipos de relaciones más comunes

Uno a uno (1:1)

Cada instancia de una entidad se relaciona con una sola instancia de otra.

Ejemplo: Un empleado tiene un contrato.

Uno a muchos (1:N)

Una instancia de una entidad se relaciona con múltiples instancias de otra.

Ejemplo: Un docente imparte varios cursos.

Muchos a muchos (N:M)

Varias instancias de una entidad se relacionan con varias de otra.

Ejemplo: Estudiantes inscritos en múltiples cursos y cursos con múltiples estudiantes.

Representación en tablas

- Las relaciones 1:N se representan agregando la clave primaria de la entidad "uno" como clave foránea en la entidad "muchos".
- Las relaciones N:M requieren una entidad intermedia (o tabla puente) con claves foráneas de ambas entidades.

Ejemplo práctico: Relación entre Estudiante y Curso:

- Un estudiante puede estar matriculado en varios cursos.
- Un curso puede tener muchos estudiantes. → Relación muchos a muchos. Se implementa una tabla "matrícula" con claves foráneas:

Entidad: MATRÍCULA

| id_matricula (PK) |

| id_estudiante (FK) |

| id_curso (FK) |

| fecha_inscripcion |

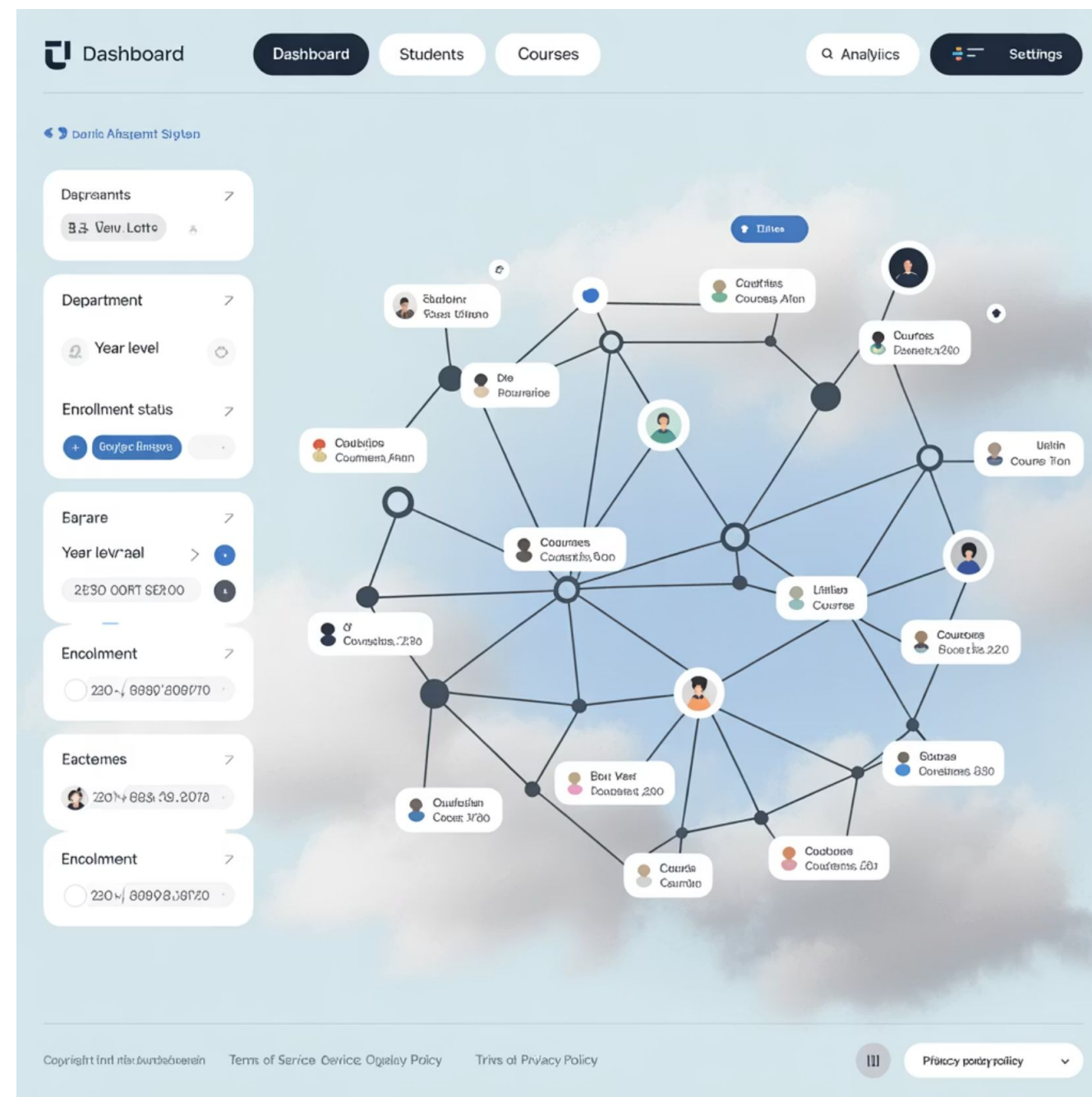
Código SQL ejemplo

```
CREATE TABLE estudiante (  
    id_estudiante INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    correo VARCHAR(100)  
);  
  
CREATE TABLE curso (  
    id_curso INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    horas INT  
);  
  
CREATE TABLE matricula (  
    id_matricula INT PRIMARY KEY,  
    id_estudiante INT,  
    id_curso INT,  
    fecha_inscripcion DATE,  
    FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante),  
    FOREIGN KEY (id_curso) REFERENCES curso(id_curso)  
);
```

Este código muestra cómo implementar una relación muchos a muchos mediante una tabla intermedia en SQL. Se incluyen claves foráneas con restricciones de integridad referencial para asegurar la validez de los vínculos entre entidades.

Contexto productivo

En sistemas de gestión académica, las relaciones entre estudiantes, cursos, docentes y evaluaciones son clave para automatizar certificados, validar matrículas o emitir reportes. Un mal diseño puede implicar duplicación de datos, errores en reportes o ineficiencia en búsquedas. Estas relaciones también son críticas para aplicar seguridad en el acceso a los datos, por ejemplo, cuando se restringe el acceso a información de cursos según el rol del usuario.



Buenas prácticas



Representar relaciones complejas con entidades intermedias claramente nombradas.



Establecer restricciones de integridad referencial para evitar datos huérfanos.



Evitar relaciones redundantes o no justificadas por los requerimientos del negocio.



Asegurar que las claves foráneas estén correctamente indexadas para mejorar el rendimiento en consultas.

Errores comunes



Duplicar atributos para representar relaciones (ej. varios campos curso_1, curso_2...).



No validar las relaciones con los equipos funcionales o usuarios finales.



No considerar la cardinalidad real observada en el negocio.



Asumir que todas las relaciones deben modelarse de forma explícita sin analizar la carga operativa que esto implica.

Diagrama Entidad-Relación (DER)

El Diagrama Entidad-Relación es una representación gráfica que permite visualizar las entidades del sistema, sus atributos, identificadores y las relaciones entre ellas. Es una herramienta fundamental para el diseño conceptual de la base de datos, y actúa como un puente entre el lenguaje técnico y el lenguaje del negocio.

Componentes del DER

- Entidades representadas como rectángulos.
- Atributos representados como óvalos.
- Relación representada como rombos.
- Líneas conectan los elementos y expresan cardinalidad (1:1, 1:N, N:M).

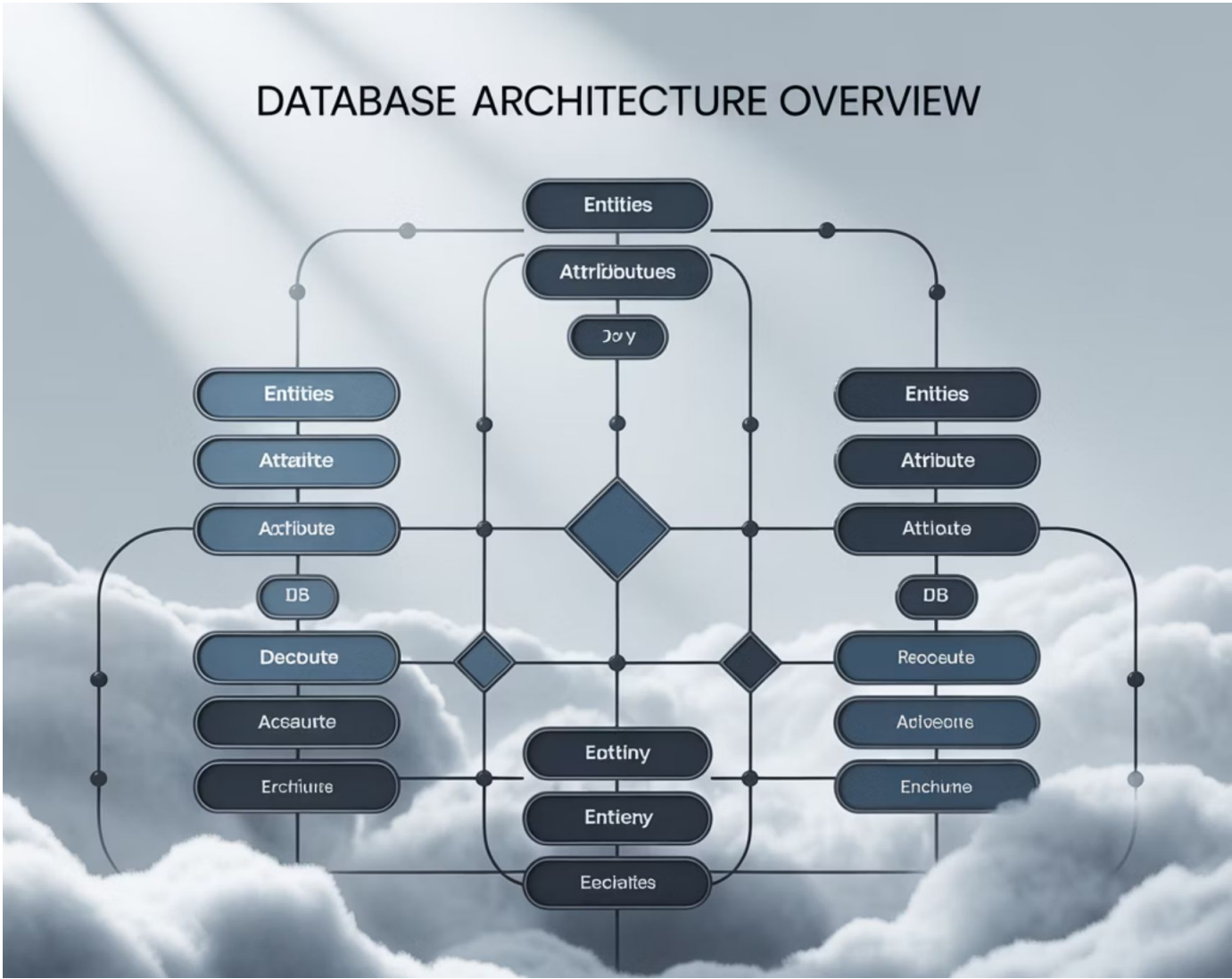
Ejemplo visual simplificado

[Estudiante]-----[Curso]

||

[nombre] [nombre]

[correo] [horas]



Aplicaciones en la práctica profesional



Validación de diseño

Validar el diseño lógico de la base de datos con usuarios no técnicos.



Trabajo colaborativo

Facilitar el trabajo colaborativo entre analistas, diseñadores y desarrolladores.



Documentación

Servir como base para documentación técnica del sistema.



Detección de errores

Detectar omisiones o redundancias antes de pasar al diseño físico.

Herramientas recomendadas para su construcción

- dbdiagram.io
- [Lucidchart](https://lucidchart.com/)
- [Draw.io](https://draw.io/)
- [MySQL Workbench](https://mysqlworkbench.dev/)
- [Diagrams.net](https://diagrams.net/)

Buenas prácticas al construir un DER



Usar nomenclatura clara, sin abreviaturas ambiguas.



Incluir todos los identificadores clave y relaciones necesarias.



Confirmar la cardinalidad con fuentes del negocio.



Actualizar el DER constantemente ante cambios en los requerimientos.

Errores comunes



Omitir entidades clave por enfocarse solo en requerimientos técnicos.



No indicar claves primarias o relaciones opcionales.



No actualizar el DER cuando cambian los requerimientos del sistema.



Usar convenciones de notación inconsistentes o poco comprensibles para el público objetivo.

Actividad guiada:

Diseñando un modelo de datos para gestión académica



Objetivo de la actividad

Diseñar un modelo de datos relacional a partir de un caso contextualizado, identificando entidades, atributos e identificadores, estableciendo relaciones y representándolas mediante un diagrama entidad-relación (DER), con el propósito de construir una base estructurada y normalizada para un sistema académico.



Situación problema

Formas parte del equipo de análisis de datos de un centro de formación técnica que está migrando desde registros manuales y planillas Excel hacia un sistema de gestión académico automatizado.

La dirección académica necesita que desarrolles la estructura base del sistema, permitiendo gestionar estudiantes, cursos y procesos de matrícula.

Esta estructura debe garantizar integridad, escalabilidad y claridad, considerando relaciones entre entidades y evitando ambigüedades o duplicación de información.

Se espera que entregues un modelo inicial en formato DER que pueda ser validado por el equipo de TI y por usuarios no técnicos.



Instrucciones



Leer y comprender la situación problema presentada.



Identificar al menos tres entidades relevantes al caso.



Definir un conjunto de atributos para cada entidad, justificando su elección.



Establecer la clave primaria de cada entidad.



Determinar las relaciones entre las entidades y su cardinalidad.



Construir un diagrama entidad-relación (puede ser dibujado a mano o con herramientas digitales).



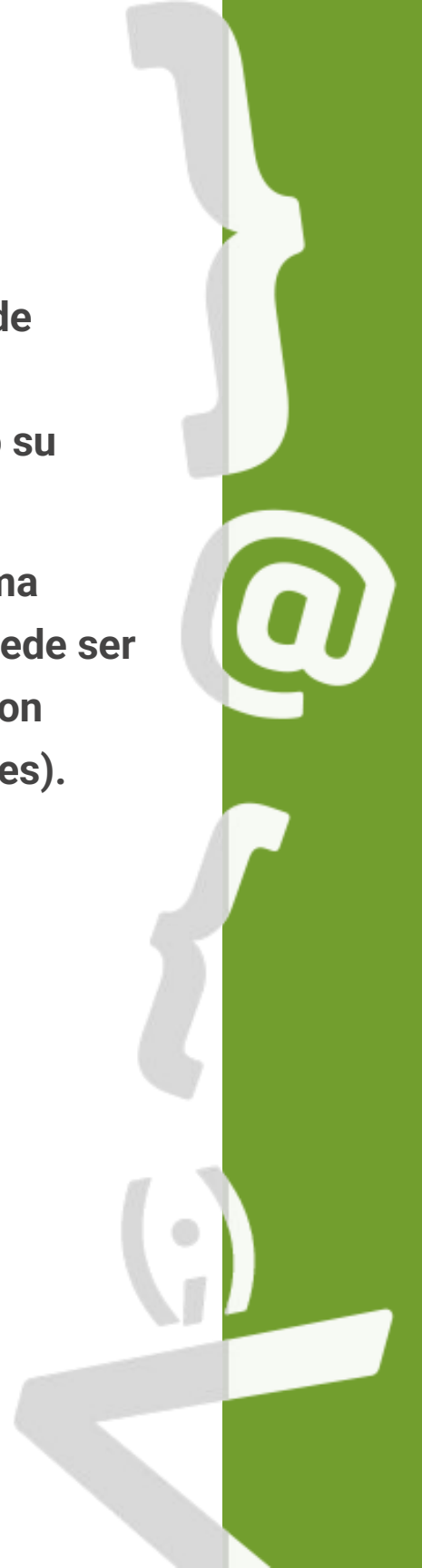
Validar en grupo si el modelo es comprensible por personas no técnicas.



Reflexionar en grupo: ¿qué decisiones fueron difíciles?, ¿qué cambiarías para adaptarlo a otro contexto?

Materiales necesarios

- Entorno de desarrollo (SQL online, Jupyter Notebook, IDE local).
- Plantilla para DER (física o digital).



Resumen

1

Modelo de datos y elementos estructurales

Comprendimos qué es un modelo de datos y cuáles son sus elementos estructurales.

2

Entidades, atributos e identificadores

Identificamos y diferenciamos entidades, atributos e identificadores en un contexto productivo real.

3

Relaciones entre entidades

Establecimos relaciones entre entidades aplicando criterios de cardinalidad y analizamos cómo representarlas adecuadamente.

4

Diagramas entidad-relación

Diseñamos diagramas entidad-relación como herramienta de validación y comunicación técnica.

5

Aplicación práctica

Aplicamos el diseño lógico de un modelo de datos en un caso contextualizado de gestión académica.



Próxima sesión...

En la siguiente sesión avanzaremos hacia la manipulación inicial de bases de datos a través de consultas básicas.

Aprenderemos a seleccionar columnas específicas con la instrucción `SELECT`, elegir la tabla a consultar mediante `FROM`, y aplicar filtros con `WHERE` para extraer subconjuntos de datos relevantes.

Esto nos permitirá comenzar a explorar los datos almacenados en las estructuras diseñadas y a formular consultas que respondan a necesidades concretas de análisis y reporte.

Preguntas de cierre

-  ¿Qué elementos mínimos debe contener un modelo de datos relacional para que sea funcional y escalable?
-  ¿Cómo se implementa en SQL una relación muchos a muchos y por qué no es recomendable hacerlo directamente entre dos tablas?
-  ¿Qué problemas podrían surgir si se omite definir la clave primaria en una tabla?
-  ¿Qué riesgos conlleva definir relaciones sin tener claridad sobre la cardinalidad entre entidades?
-  ¿En qué momentos de un proyecto de desarrollo es crucial contar con un DER validado?

Referencias bibliográficas

- Coronel, C., Morris, S., & Rob, P. (2020). Database Systems: Design, Implementation, & Management (13th ed.). Cengage Learning.
- Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems (7th ed.). Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.
- MySQL Documentation. (2024). The MySQL Developer Guide. <https://dev.mysql.com/doc/>
- Draw.io. (2024). Diagrams.net User Guide. <https://www.diagrams.net/doc/faq>